Acknowledgement

I express my sincere gratitude to the management of Mar Athanasius College of Engineering, Kothamangalam, for providing the academic environment and facilities required to undertake this mini project. The opportunity to study data preparation, machine learning, software design, and responsible review workflows in one project has been an important part of my learning.

I am grateful to my project guide, Dr.Manu John, for guidance, constructive feedback, and encouragement during the planning and development of the AI Forensic Insider Risk Review System. I thank Dr.Sonia Abraham, Head of the Department of Computer Applications, and the project coordinators and faculty members for their support and academic direction.

I also acknowledge the creators and maintainers of the CERT Insider Threat Test Dataset and the developers of the open-source libraries used in this project. The public synthetic dataset made it possible to explore a sensitive research problem without using real employee records. Finally, I thank my family and friends for their encouragement throughout the project.

Abstract

Insider-risk review is difficult because potentially relevant activity is distributed across multiple systems, the number of routine events is large, and known incident examples are rare. This mini project presents Forensio, a local-first review system that aggregates logon, removable device, file, email, and web activity into employee-day features and uses four different model perspectives to prioritize records for human inspection. The implementation combines a Python data and scoring pipeline, a FastAPI service, and a React review dashboard; a Streamlit interface remains available as an alternative.

The model-development study uses the CERT r4.2 synthetic insider-threat dataset. Its saved labelled table contains 330,452 employee-days for 1,000 synthetic users, of which 1,364 employee-days overlap a listed incident interval under the project's labeling rule. Activity events are cleaned and aggregated by user and calendar day. A grouped train/validation/test design keeps users disjoint between partitions. The notebook compares Logistic Regression, Decision Tree, Random Forest, and Isolation Forest, selects variants and a review capacity using validation data, and evaluates the selected operating point on a held-out test partition. In the saved one-percent review-capacity evaluation, the Random Forest has precision 0.144, recall 0.329, F1 0.200, and average precision 0.158. The dashboard separately combines within-dataset percentile ranks and applies an employee-level queue rule.

The system is a research and demonstration prototype, not an investigative finding engine. A high rank is neither a probability nor evidence of misconduct. The dataset is synthetic, labels are incident-window proxies, queue rules are relative to a selected dataset, and the current features do not model employee roles, schedules, or context. Human review, corroborating evidence, access controls, and documented governance are therefore essential before any operational use.

List of tables

List of figures

Contents

1. Introduction

1.1 Background

Modern organisations generate activity records through identity providers, endpoints, file services, email systems, and web gateways. These records are useful for security operations, but their volume and fragmentation make manual review difficult. A security analyst may need to correlate a sequence of events across several sources before deciding whether it warrants escalation. The challenge is not simply to detect a single unusual event; it is to organise context in a way that supports a careful and reproducible investigation.

Insider-risk analysis concerns activity associated with trusted accounts and authorised access. A meaningful review must distinguish between a statistical signal and a conclusion about a person. Legitimate work can produce unusual activity, while harmful activity may resemble normal operations. Differences in job role, work schedule, travel, project deadlines, and system configuration can all affect the observed event pattern. For that reason, a decision-support tool should make its inputs and ranking logic understandable and should preserve human authority over all consequential decisions.

This project implements a prototype called Forensio. It transforms five activity logs into employee-day counts, scores those feature rows using four model families, and displays a ranked review queue. It is designed to demonstrate a technical workflow on the CERT r4.2 synthetic dataset. It is not a production surveillance platform, does not inspect message or file content, and does not establish intent.

1.2 Problem statement

The project addresses the following bounded problem: given a local folder containing the five supported activity CSV files and a previously trained model bundle, produce an employee-day and employee-level ranking that helps an analyst decide which records to review first. The system must integrate disparate sources, handle large CSV files with bounded-memory processing, apply the same feature contract used in training, expose clear model context, and avoid presenting a score as proof.

The study also evaluates whether a set of supervised and unsupervised models can rank synthetic incident-window employee-days above other days when users are kept separate across development and test partitions. A low base rate makes accuracy alone unsuitable as the primary measure. The study therefore reports average precision and capacity-based precision, recall, and F1 alongside the confusion counts.

1.3 Objectives of the project

The objectives are to:

1. examine the supported activity data and its quality;
2. build a repeatable employee-day aggregation and labeling workflow;
3. compare four distinct model families under user-grouped evaluation;
4. build a local scoring service that reuses the trained feature schema;
5. create a dashboard for selecting data, following analysis progress, and reviewing ranked employees and activity details;
6. document evaluation limits, privacy considerations, and responsible-use controls.

1.4 Scope of the project

The project scope includes synthetic logon, device, file, email, and HTTP event records; aggregation at one calendar day per user; the feature columns persisted with the trained bundle; offline model evaluation; local scoring; and a human-readable review interface. Optional employee-name lookup files are supported by the local scoring code, but names are not required for model input.

The current scope excludes live collection agents, enterprise identity integrations, cross-organisation model transfer, streaming analysis, automated disciplinary decisions, legal attribution, message-body or file-content inspection, calibrated probability estimates, and production authentication or role-based access control. A dashboard rank is relative to the data batch being scored. The offline employee-day test results do not validate the dashboard's employee-level status rule.

1.5 Project domain and technology stack

The project combines data engineering, applied machine learning, information visualization, and digital-forensics workflow concepts. Python and pandas perform input handling and aggregation; scikit-learn supplies estimators and metrics; joblib persists the model bundle; FastAPI exposes a local HTTP interface; and React, JavaScript, Vite, and CSS provide the browser dashboard. Streamlit is retained as an alternative interface. CSV files are used for both inputs and result exports; the prototype does not require a database.

Table 1.1: Project components and their responsibilities

Component	**Technology**	**Responsibility**
Data preparation	Python, pandas, NumPy	Clean activity records, aggregate features, and align model columns.
Model development	scikit-learn	Fit and evaluate supervised and unsupervised estimators.
Model persistence	joblib	Store estimators and the feature-column contract for local reuse.
Service layer	FastAPI, Pydantic	Validate requests, manage analysis endpoints, and return JSON results.
Web interface	React, Vite, CSS	Select log folders, display progress, search and filter the review queue.
Alternative interface	Streamlit	Provide a Python-only local demonstration path.
Input and outputs	CSV	Interchange activity logs, scores, and report tables.

1.6 Organisation of the report

Chapter 2 reviews insider-risk analytics and relevant evaluation principles. Chapter 3 describes the dataset, preprocessing, visualization, algorithms, pipeline, feasibility, and operating environment. Chapter 4 presents model construction, software architecture, requirements, data design, and interface design. Chapter 5 reports the saved model evaluation and discusses its limits. Chapter 6 describes the local deployment and analyst workflow. Chapter 7 summarises the repository history. Chapters 8 and 9 give the conclusions and future work. Chapter 10 contains reproducibility information, requirements, test cases, data definitions, responsible use guidance, and source listings. Chapter 11 provides references.

2. Supporting literature

2.1 Literature review

Insider-risk detection is difficult because legitimate and risky activities can share the same systems and permissions. Research therefore examines both user behaviour and the context needed to interpret it, rather than treating a model score as proof of intent. The CERT Insider Threat Test Dataset provides synthetic scenarios and activity records for repeatable experiments [**?**]. Synthetic data avoids exposing real employee records during development, but results on one generated release do not establish performance in a real organisation.

Rahman, Al Naeem, and Abubakar study unintentional-insider readiness using CERT r4.2 activity data and a nonlinear autoregressive model with exogenous inputs (NARX). They describe access, motivation, and action as dimensions of insider behaviour and report a best-partition R^2 value of 0.97 for their proposed formulation [**?**]. This work is relevant because it explicitly considers unintentional rather than exclusively malicious behaviour. Its target formulation and reported statistic differ from the employee-day classification and review-capacity metrics used in this project, so the figures should not be compared as if they measured the same task.

Al-Shehari et al. investigate an anomaly-based Isolation Forest model for the class-imbalance problem in insider-threat data, using the CERT benchmark and varying the model's contamination setting. They report 98% accuracy for their proposed approach [**?**]. The study motivates including an unsupervised anomaly detector alongside supervised classifiers. However, accuracy alone does not describe how many rare positive cases are found or how many reviewed records are positive. For an imbalanced review queue, precision, recall, average precision, and results at a specified review capacity provide additional information.

Insider-risk prioritisation is related to, but distinct from, digital-forensics-based attack detection. Dimitriadis et al. present Fronesis, which correlates digital artifacts with MITRE ATT&CK techniques and Cyber Kill Chain phases using an ontology and rule-based reasoning; they demonstrate the approach with a phishing scenario [**?**]. This work highlights the importance of preserving and correlating evidence. The present project has a narrower analytical scope: it ranks synthetic employee-day records and directs a reviewer to source activity, rather than detecting an ongoing attack or reconstructing a forensic timeline.

The project follows a common feature-engineering pattern: transform event-level data into a table whose rows represent a user and a calendar day. Aggregation reduces the number of records and makes multi-source activity easier to compare. Count and timing features can reveal volume changes or activity outside a fixed interval, but they lose event ordering and detailed context. A day-level table cannot reconstruct a forensic timeline. The system therefore treats the aggregate as a navigation aid and directs an analyst back to original records for validation.

The algorithm set deliberately includes contrasting assumptions. Logistic Regression is a linear supervised classifier and provides a relatively simple baseline. A Decision Tree expresses threshold relationships through a sequence of splits but can overfit if unconstrained. A Random Forest averages many trees to reduce the instability of a single tree. Isolation Forest isolates observations through random feature and split selection, assigning higher anomaly scores to observations isolated in fewer splits [**?**]. The estimator used in this project is implemented by scikit-learn [**?**]. These algorithms do not produce interchangeable quantities: a class probability, a decision function, and an anomaly score have different semantics. This project converts each output to a percentile rank within the current scoring batch before combining dashboard scores.

Rare labels and high class imbalance are central evaluation concerns. When a positive example is a small fraction of a test set, a model can achieve high accuracy by ranking nearly all records as negative. Precision-recall measures are more informative for a review queue because they describe the fraction of selected records matching known positive labels and the fraction of known positives retrieved. Average Precision summarises precision over score thresholds. A fixed-capacity queue also corresponds to an operational constraint: an analyst can review only a limited number of records in a given period. Evaluation must specify whether capacity selection was performed on validation data or the test set.

User-level leakage is another important issue in behavioural data. Repeated daily records from one person share stable characteristics. If records are randomly distributed across training and testing, the model may benefit from seeing a user's other days before evaluation. The notebook therefore uses stratified group splitting with the user identifier as the group. The identifier is not a model feature; it is used to keep users in one partition.

Responsible monitoring practices require a clear purpose, access limits, proportionality, retention controls, and a review process. Security frameworks such as NIST SP 800-53 describe system and information security controls, while NIST SP 800-61 describes incident response practices [**?**, **?**]. These documents are not specific model-governance standards, but they provide useful control contexts for access, logging, incident handling, and evidence preservation. A risk score alone should not trigger an adverse action.

2.2 Literature review summary

Table 2.1: Literature themes and their application to this project

Theme	**Practical implication**	**Project response**
Synthetic and unintentional-insider research	CERT supports repeatable experiments; the unintentional-insider study uses a different target formulation.	Use CERT r4.2 and limit conclusions to this project's employee-day labels and evaluation protocol.
Anomaly detection under class imbalance	Isolation Forest offers a label-independent scoring perspective; accuracy alone may hide minority-class performance.	Compare it with supervised models using ranking and fixed-capacity metrics.
Digital-forensics-based detection	Correlating artifacts with attack techniques can support incident detection and evidence interpretation.	Treat the dashboard as a prioritisation aid and require analyst validation against original records.
Event aggregation	Simplifies modelling but removes sequence and context.	Aggregate to employee-day counts and preserve source records for follow-up.
Rare positive labels	Accuracy can conceal low precision and missed cases.	Report AP and capacity-based precision, recall, F1, and confusion counts.
Repeated observations	Random row splits can leak individual patterns.	Keep employees disjoint across training, validation, and test partitions.
Human investigation	A ranking or a detected artifact is not, by itself, a finding about intent or misconduct.	Display review language and require independent analyst verification.

2.3 Findings and proposals

The reviewed papers address complementary problems: unintentional insider readiness, anomaly scoring under class imbalance, and forensics-based correlation of evidence during an ongoing attack. Their targets, methods, and reported metrics differ, so their published results are context rather than a direct benchmark for this implementation. Together with the CERT dataset documentation, they motivate four design decisions. First, the project uses a multi-source daily representation rather than treating each event as an independent case. Second, it compares model families instead of presuming that one algorithm is universally suitable. Third, it separates model selection from final test reporting and reports review-capacity metrics for rare labels. Fourth, it

describes the interface as a review aid and makes the relative nature of the scores explicit.

The literature also identifies gaps that remain open. A future evaluation should include time-based testing, additional synthetic releases, calibrated and subgroup-aware analyses where ethically and lawfully appropriate, and structured analyst feedback. Before any real-world use, an organisation would need to define a lawful purpose, consult relevant stakeholders, assess privacy and employment impacts, set retention limits, control access, and validate the complete workflow rather than only the estimator.